Coordinating Dynamically Updating AWS Website Hosting

Documentation for Making Github Automatically Update AWS Site

Prepared for Newly Hired Engineers

Created by Vikram Balaji

This Corporate Instruction document (or “Instruction Document”) states the terms and conditions that our newly hired engineers must abide by when using our virtual machine resources.

NOW, THEREFORE, In consideration of the mutual covenants and promises made by the parties hereto, the Employee and the Employer (individually, each a “Party” and collectively, the “Parties”) covenant and agree as follows will be required of them in the workplace.

Due to the graciousness of Amazon’s AWS services, next generation computing technology is on our fingertips. It will be expected that all of our engineers in the workplace learn and embrace our virtual machines to utilize them to their full potential. Those who do not abide by our strict rules for VM security and safety will be sent to our second corporate worksite in Tel Aviv for on-site retraining. The following instructions are about how to recreate container instances for Ubuntu. If you are an IT agent at our company, please follow these instructions to recreate the environment in the case we must do so.

1. Initialize Secured Website

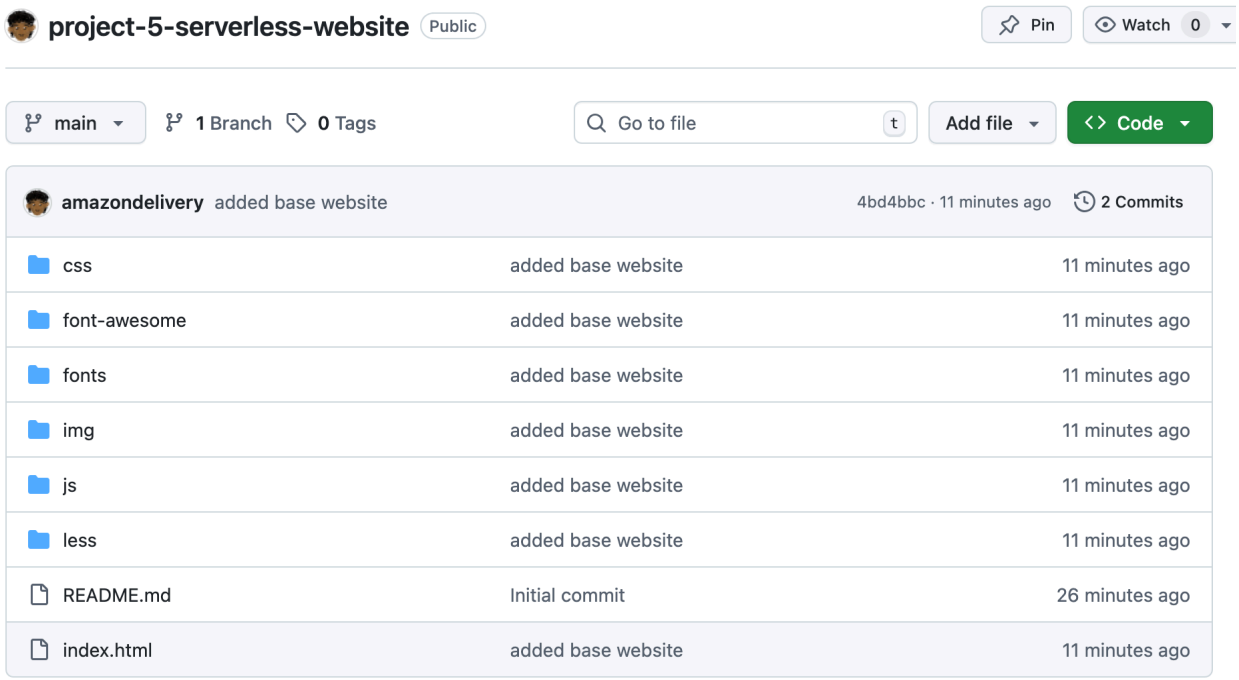
Make sure that your website is secured. In this organization, we used Amazon’s S3 buckets to host the website files and serve them when needed. S3 buckets are a great option for this because they use high powered computers to quickly serve whatever is needed. Next, we used Namecheap to buy our domain and then route it to the S3 bucket to show the website when you go to that domain, paired with the help of using route 53 to make a url for our bucket to serve the files. For making sure that the connection to the website is secured, we use AWS Certificate Manager to generate a certificate for our website. Finally, we use AWS Cloudfront to make sure that the website is cached on servers around the world and can easily be loaded to a user. Make sure this secure website is set up before we continue.

2. Create Github Repository

First, go to your terminal and then make the directory for your github repository by typing in `mkdir {repo_name}` Then `cd` into it, then do `git init`. Go to github.com and create a repository for your project. On your terminal, type `git add –all`, `git commit -m “initial commit”`, then `git remote add origin https://github.com/[username]/[repository_name].git`. Finally, `git push -u origin main`. You now have connected your local folder to your github repository.

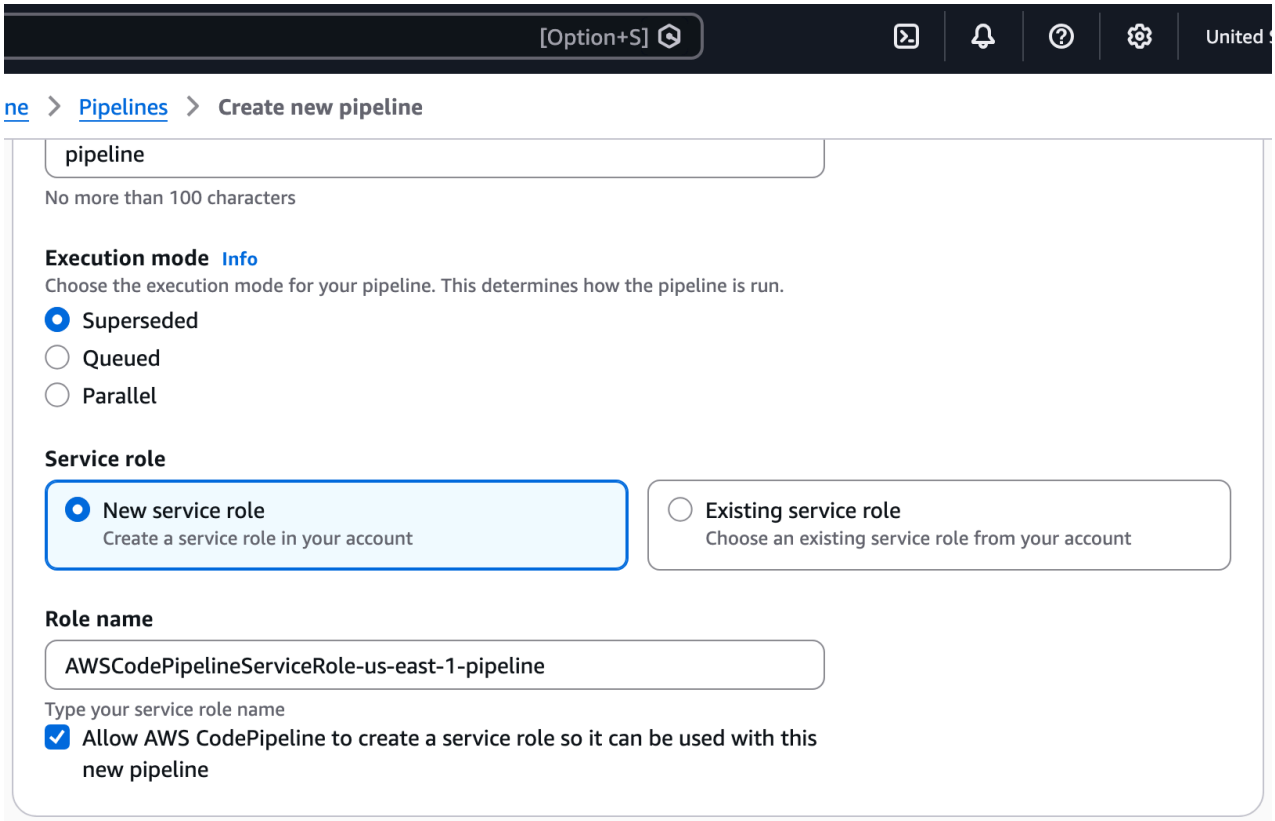
3. Add Your Website Files

Add your website files to the folder, then add, commit, and push them to your Github repository.



4. CodePipeline

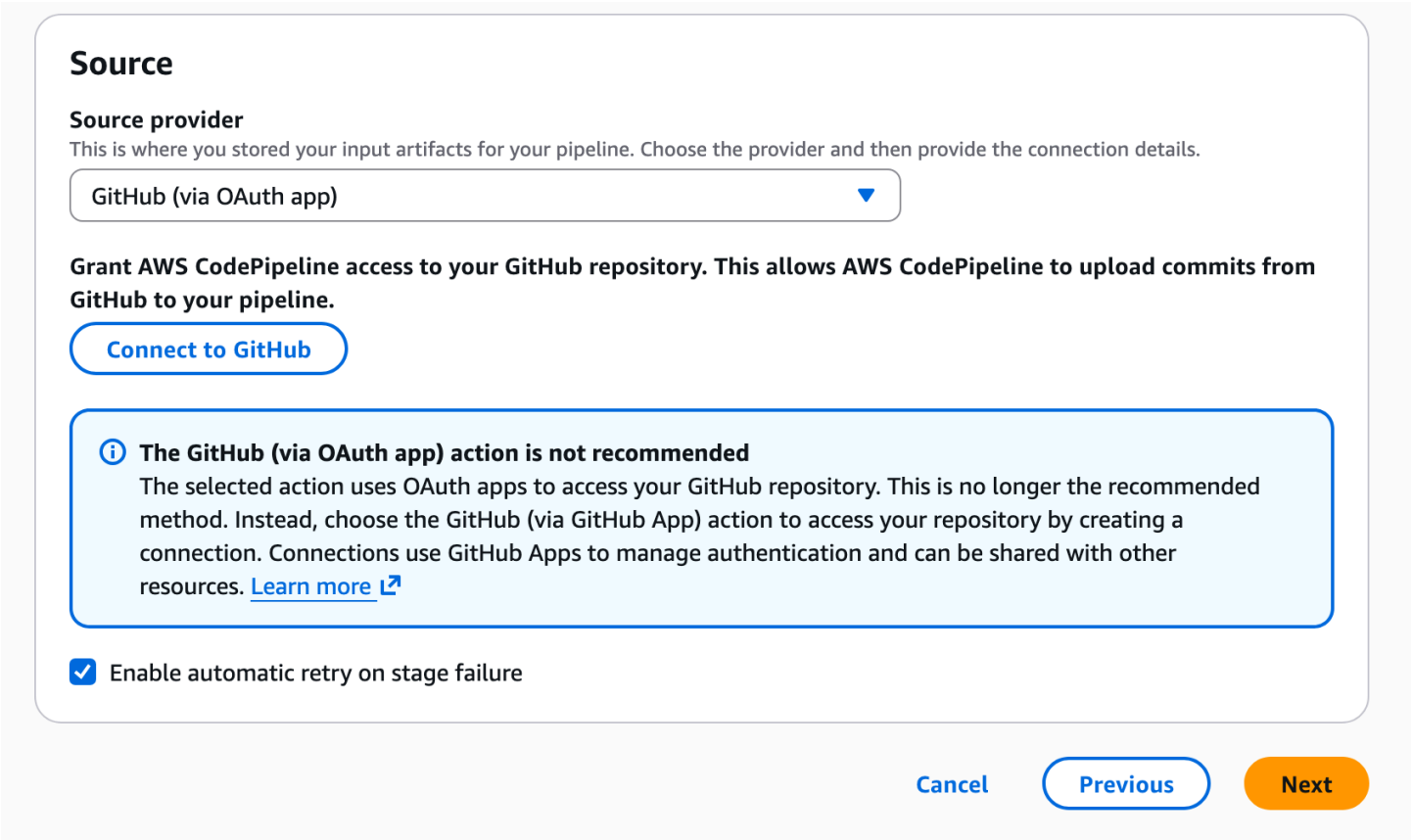
Navigate to the AWS CodePipeline console and begin the creation of a new code pipeline. Select “Build custom pipeline.” Choose the execution mode as Superceded so that only the latest push to github actually gets deployed, instead of them being deployed in a row.



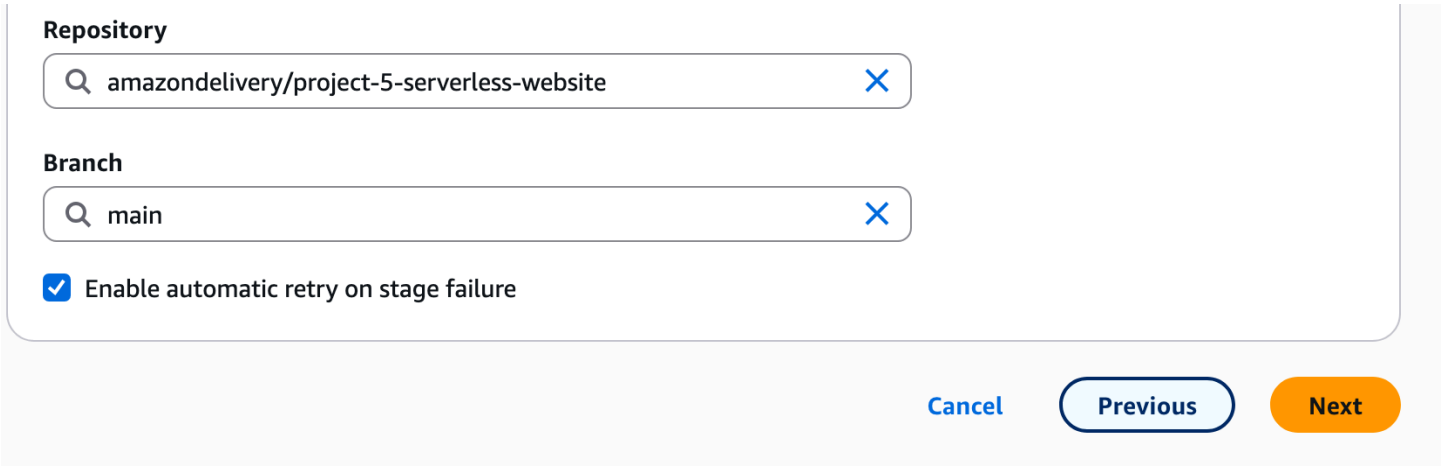
This document is meant to serve as a reference. You will be liable for excessive use of company credits.

5. Source

For the source, enter Github via OAuth application, then log in and authorize the connection.



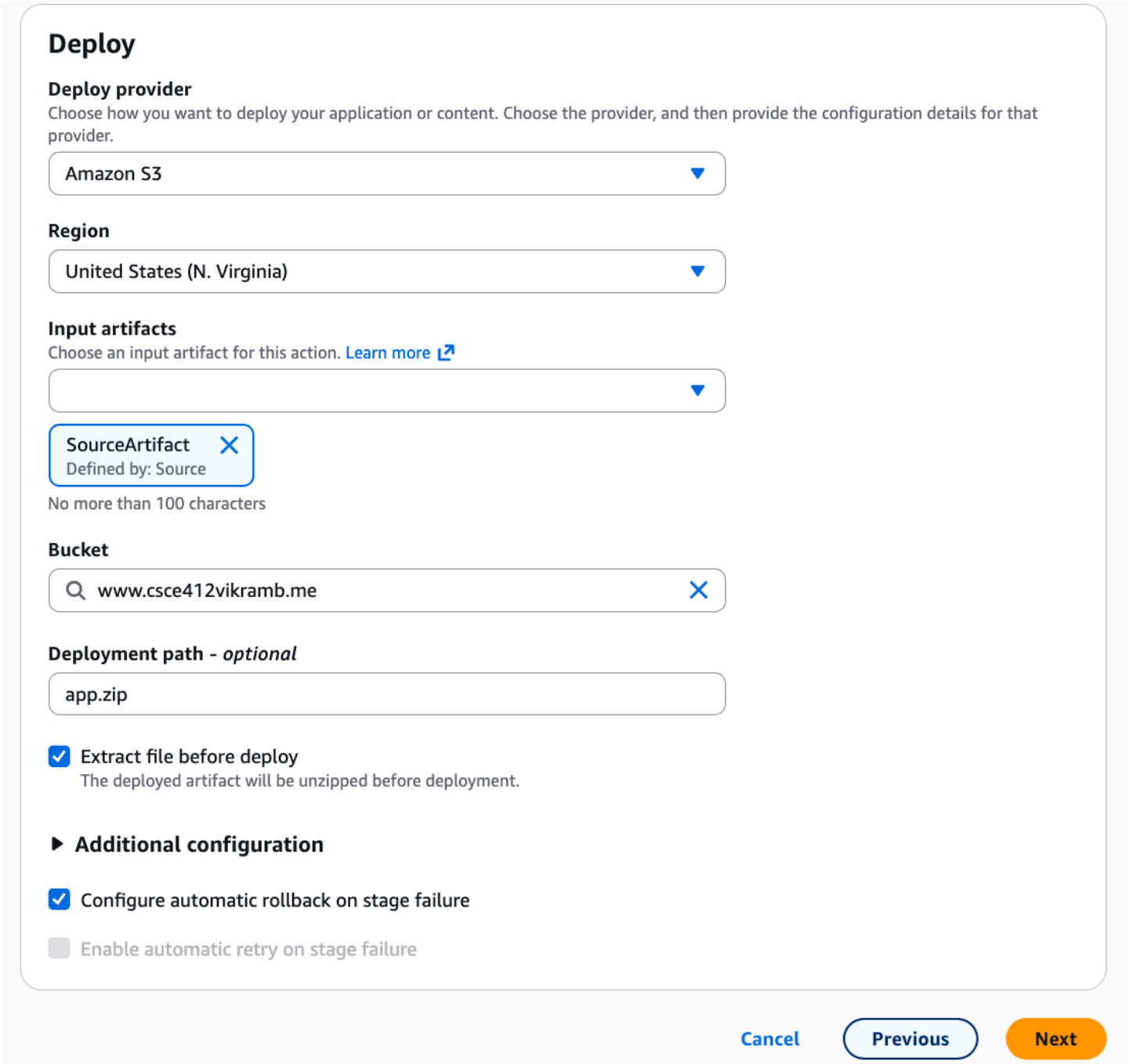
Then identify the repo and branch that you want to connect.



5. Selecting Bucket Deployment

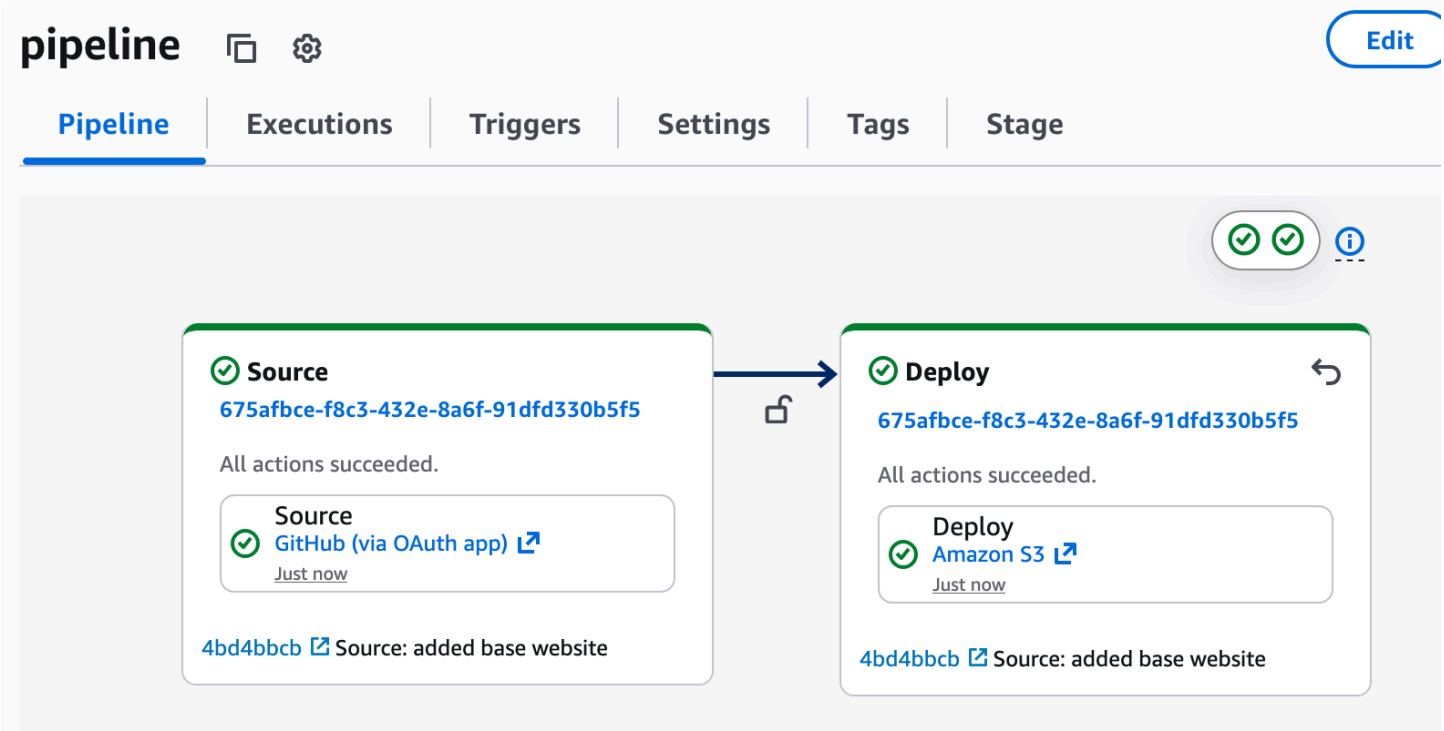
Now, skip the build and test parts of the CodePipeline creation step and then go to the deployment stage. Select S3 as your provider, and select the bucket that hosts your website files. Make your deployment path something like “app.zip” and then select Extract before deployment, as shown below.

This document is meant to serve as a reference. You will be liable for excessive use of company credits.



Now, click create and it should show you a flow chart of the pipeline.

This document is meant to serve as a reference. You will be liable for excessive use of company credits.

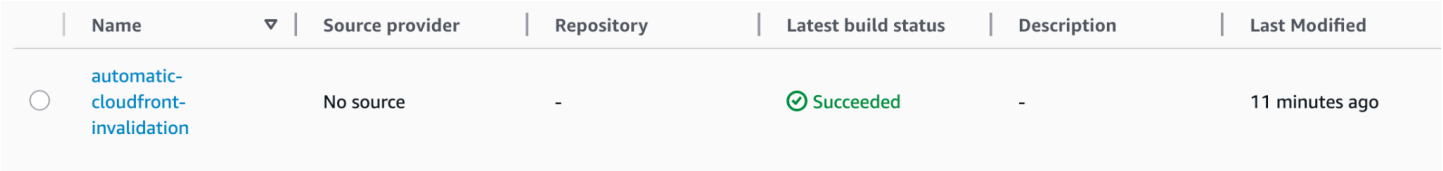


Now, every time you push a commit to github, the repository will automatically update the website stored in the bucket

6. One Final Problem

The final problem is that the cache of the website, stored in CloudFront, is still served to users. This means that an older version of your website will be shown to potentially millions of users around the globe until that cache is invalidated. To invalidate it the moment that you deploy, we will use CodeBuild. We go to the CodeBuild AWS portal and then click “Create Project.” Make sure to select that this will be a lambda function. This ensures that it runs quickly and without too much processing power required. Keep all of the other options to their default and scroll down to the build command and then paste this in:

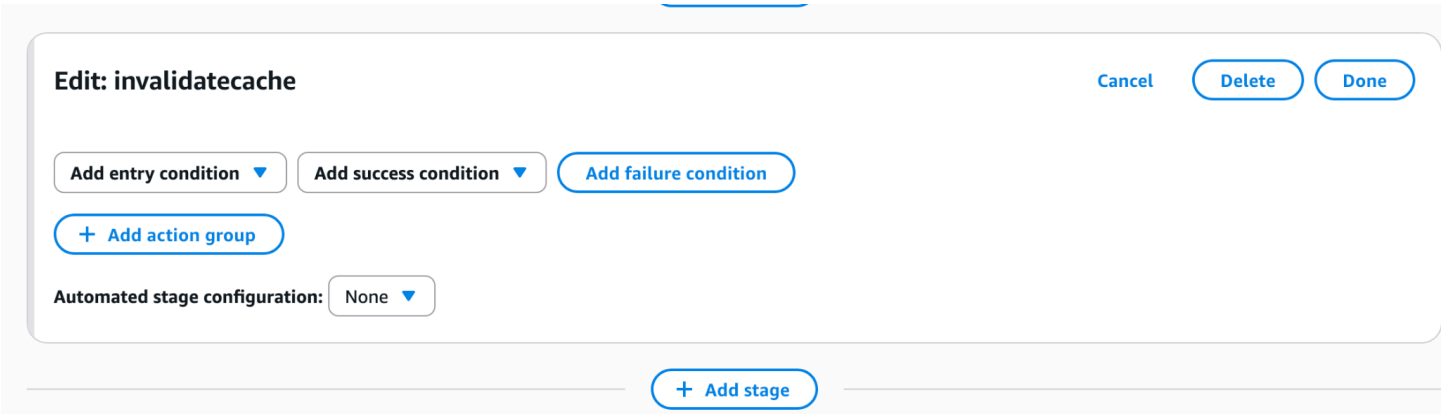
```
version: 0.2
phases:
 build:
  commands:
   - aws cloudfront create-invalidation --distribution-id DIST_ID --paths "/*"
```



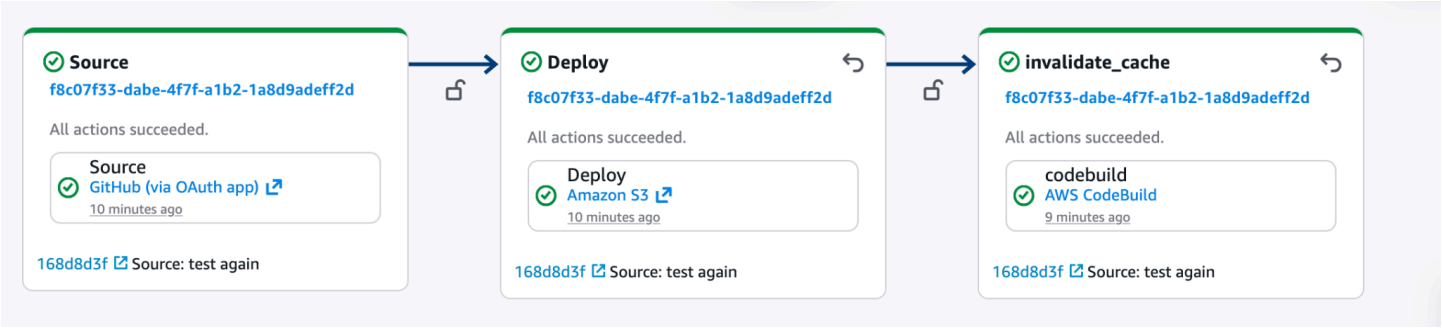
Name	Source provider	Repository	Latest build status	Description	Last Modified
automatic-cloudfront-invalidation	No source	-	Succeeded	-	11 minutes ago

This document is meant to serve as a reference. You will be liable for excessive use of company credits.

where DIST_ID is the distribution ID of your Cloudfront object. Now, you click Create and then you have to modify some permissions to make sure you are able to actually run this on your Cloudfront object. Go to IAM by typing it in the search bar. In the roles tab, look up the AWSCodeBuildDeveloperAccess permission and enable it. Then also enable cloudfront:CreateInvalidation for your CodeBuild service. Finally, in your pipeline, click edit, scroll down, and click add stage.



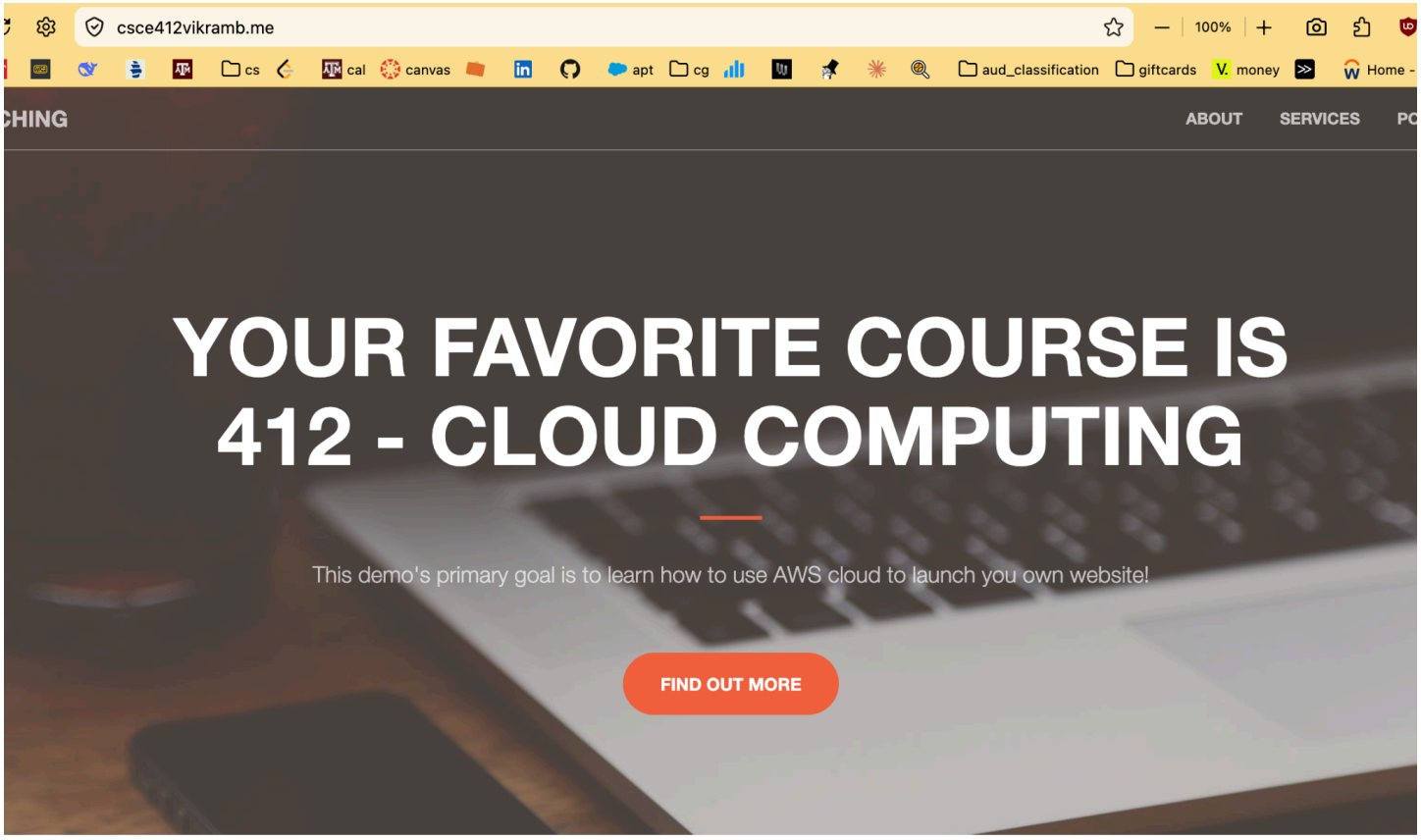
In the action provider section, add codebuild, select the source artifact, then the project name, and leave all the other ones at the default setting and click done. Save, and now your Cloudfront invalidation should be automatic.



Your pipeline should look like this now.

Assuming you have a local version of the repository on your computer, you are now able to update the website by simply pushing to the remote git repo. To do this, open the website on a code editor like VSCode. For example, this is how the website looks right now.

This document is meant to serve as a reference. You will be liable for excessive use of company credits.



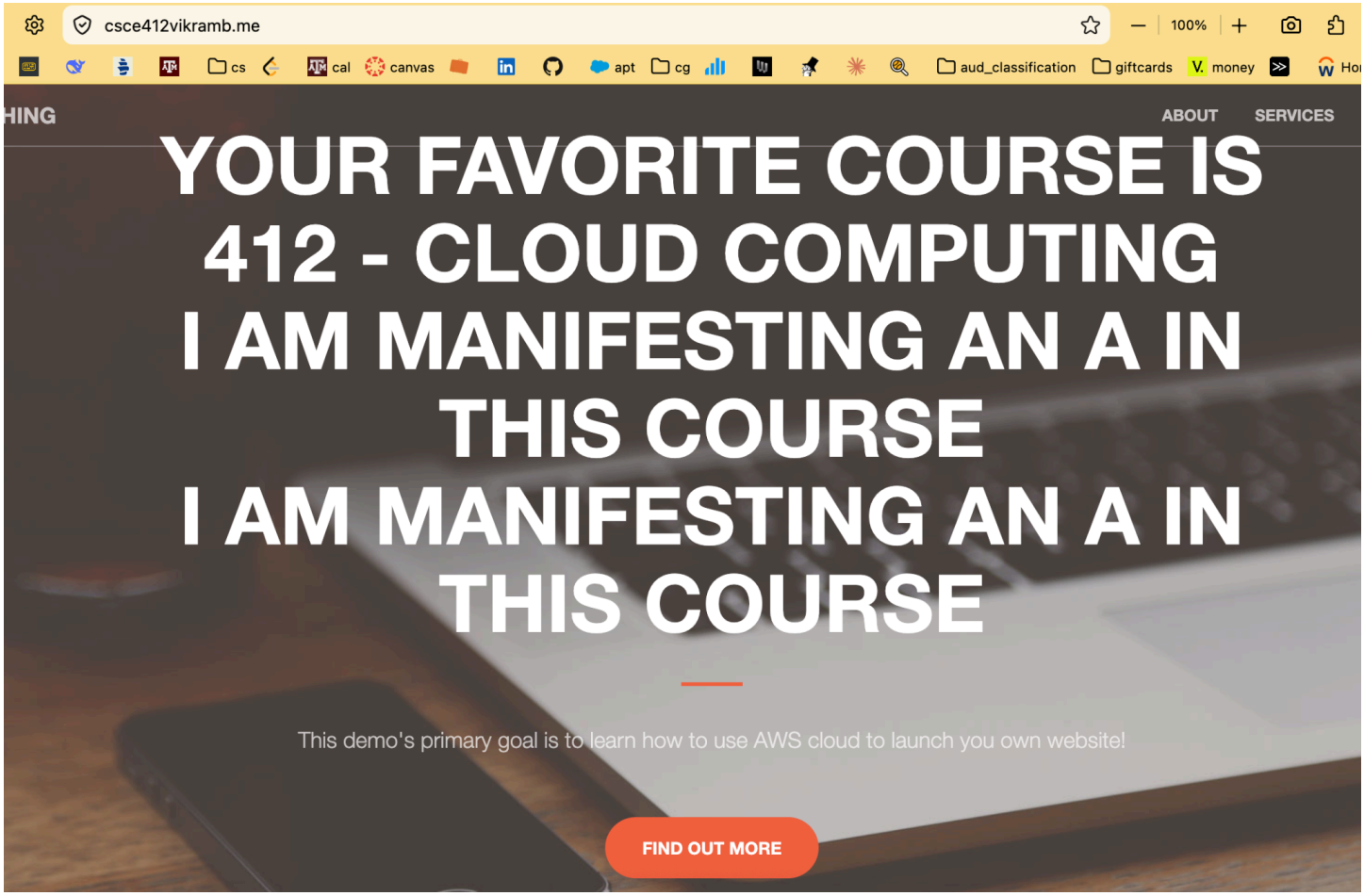
I will add the following lines of code, and then I will commit and push to Git.

```
<div class="header-content-inner">
    <h1>Your Favorite Course is 412 - Cloud Computing</h1>

    <h1>I am manifesting an A in this course</h1>
    <h1>I am manifesting an A in this course</h1>

    <hr>
    <p>This demo's primary goal is to learn how to use AWS
```

This document is meant to serve as a reference. You will be liable for excessive use of company credits.

```
vikram@le-macbook-air-4 project_5 % git add --all
vikram@le-macbook-air-4 project_5 % git commit -m "added manifestation"
[main f27d541] added manifestation
 1 file changed, 4 insertions(+)
vikram@le-macbook-air-4 project_5 % git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 359 bytes | 359.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/amazondelivery/project-5-serverless-website.git
   168d8d3..f27d541  main -> main
vikram@le-macbook-air-4 project_5 %
```

We can see, after around 30 seconds which is how long it takes for our CodePipeline to work, that the website is now automatically updated.



This document is meant to serve as a reference. You will be liable for excessive use of company credits.